CASE 1 & CASE 2 ANSWERS

Prepared by Kevin Kahumba

Odoo v17.0 Technical Assignment Answers for case 1 & 2

Case 1 Answers

How it works (default Odoo v17.0 Enterprise)

In Odoo v17.0 the default the functional flow is like:

Frontend

- The customer has a configured account for accessing Odoo webshop.
- The customer logs in to the account and navigates to the shop and selects items to put in the cart.
- Customer checks out and navigates to the payment page where, using the wire bank transfer, details are given on how to pay (i.e., pre-configured in Odoo by the company), usually the sale order as reference, bank account of the company, and amount in the cart to pay.
- The customer makes the payment using a third party (i.e., their payment apps) then waits for the company to deliver the items.

Backend

- The company is using wire transfer where they basically set up bank details for payment, which will appear on the payment page on the portal when the customer is at the checkout page.
- Some configurations (i.e res.config.settings) must be set:
    - *Invoice Online Payment* - This setting allows customers to pay their customer invoices online from the customer portal.
    - *Batch Payments* - This setting lets Odoo group multiple customer or vendor payments into one batch.You can select multiple payments, can create one batch payment, Odoo gives the batch a reference.The batch reference can help match the bank transaction during reconciliation. All payments in one batch must use the same payment method. Typical use of batch payments is: depositing multiple checks together or cash payments,grouping payments for easier reconciliation or payment files like SEPA or NACHA where applicable.
    - *Auto invoice* - It allows Odoo to automatically generate an invoice when an online payment is confirmed. It is connected to the sales invoicing configuration.It works best when the invoicing policy is **Invoice what is ordered** and If the invoicing policy is **Invoice what is delivered**, Odoo does not allow the Automatic Invoice feature, because the invoice depends on delivered quantities.

- The company, depending on the agreement with the bank, has a way to receive payment statements made on the account.
- The company using Odoo has auto invoice enabled so that invoices can be auto-generated in relation to sale orders.
- Internally, the accountant receives the transaction statements from the bank and manually records the payments, linking them to invoices and setting them to paid, draft, or pending full payment depending on how the customer paid or payment terms set for a customer.
- If the company has set up Odoo Enterprise and has all the relevant Enterprise modules, then they can take advantage of account_bank_statement_import_ofx or account_bank_statement_import_csv depending on what formats of statement the bank supports. This allows the accountant to upload these files so that they can be processed by Odoo. The OCA bank import repo has similar and more modules to support this if it's a Community Odoo setup.
- If payment is partial, follow-up is done using Odoo, i.e., message chatter.
- If payment is in full, then after it's recorded, the sales team can now set the Quotation to a Sales Order and begin sourcing the items for delivery with the warehouse team.
- Automation can be done using server actions and cron jobs where you can set up an incoming email to handle statements from the bank and fetch them periodically, then have Odoo v17.0 server actions using Python code process the OFX, CSV files and possibly record the payment transaction, then payment, validate the invoice and post, and set the sale order to done/sale state.

Customizations

Yes, we will need customizations where we can use default Odoo with the use of Odoo v17.0 Enterprise account_bank_statement_import_ofx and account_bank_statement_import_csv modules and configuring an incoming server to handle email attachments from the bank where they send bank statements periodically, which can be discussed based on urgency. Then from there we can set up server actions and a cron job which can execute Python code to handle the attachments once fetched, with custom fields using Odoo Studio, e.g., *datetime* like Last Fetch Period, *booleans* like bank statement has been processed and Auto process payment Orders which can be set in Invoice settings and once enabled will be used to allow the automatic processing of the sales process.

Impact when the company grows and the number of orders will go 10x the current volume

Some of the major impacts are:

Main risks

- More orders to verify.
- More bank statement lines to reconcile.
- More customer payment mistakes.
- More manual work for accountants.

- More chances of delayed deliveries.
- Higher risk of duplicate processing.
- More lock contention i.e transaction or process deadlocks and performance pressure if automation is poorly written.

Mitigation

- Automate the repetitive payment checks.
- Use bank statement imports or bank synchronization.
- Use payment references strictly.
- Use scheduled actions or queue jobs for background processing.
- Use clear error notifications.
- Let humans handle only exceptions.
- Avoid processing large files during peak business hours.
- Use batching for large statement imports.
- Add indexes or optimize domains if custom searches become heavy.

If there are things he should avoid (Note: the consultant is not a tech person, so perhaps you have some language he understands? But our Project Leader understands dev, so also give links to code lines (if needed, of course))

Things to avoid:

Manual dependency

- Do not rely on one accountant manually checking every payment.
- Manual processing becomes a bottleneck as volume grows.
- People can be unavailable due to leave, illness, or workload.

Over-customization

- Do not build custom code for features already handled by Odoo.
- Use default Odoo payment, accounting, import, and reconciliation features first.
- Add custom code only where the bank or business process requires it.

Weak payment references

- Do not allow vague bank references.
- The payment instruction should clearly request the customer to use the sale order or invoice reference.
- Bad references make matching harder.

Duplicate processing

- A bank statement file should never be processed twice.
- A payment should not create duplicate payments or duplicate reconciliations.
- Use tracking fields, imported statement references, or unique identifiers.

Direct purchase/delivery automation without payment certainty

- Do not trigger delivery only because a customer says they paid.
- The bank statement, payment provider, or payment transaction must confirm it.

Single points of failure

- Avoid depending on one staff member.
- Avoid relying on only one payment method.
- Avoid running everything on a single fragile technical process with no monitoring.
- Add fallback payment providers and clear exception handling.

How much time you need to give me an answer?

Time required to give an answer to the consultant is ~1 hour and a half, where I explain the problem and possible solutions, and with the assistance of the project manager we can break it down to something like below:

- Investigate issues ~1 hour and a half.
- Research solution options: best solution, architecture of solution, distribution of tasks ~1 hour and a half.
- Implementation of solution ~3 hours.
- Code Review & Testing ~1 hour.
- Deployment ~40 minutes.

NB: *Why I broke it down into approximation is that issues can't always be straightforward such that you can predict the exact period, but experience can aid in approximating time and breaking down the answer into a complete solution for the consultant to understand the whole picture and for the project manager to express it into phases.*

Is there customization needed, what kind of customization (Studio and/or module) and how much time do you need for this? He wishes to get a detailed drilldown of estimated time spending

Yes, customizations are needed, where we can customize using default Odoo v17.0 Enterprise features or automation through implementing the payment provider bank REST API for handling C2B payments modules. You can see below for a proper breakdown of implementation. The breakdown can be done in phases like below:

- Investigate issues and reporting ~1 hour and a half.
- Research solution options ~2 hours:

- ❖ Select best solutions from options (i.e., any discussions with team, bank in relation to API services and docs; communication with the relevant banks may not always be fast – it can take a couple of hours to days, but in this case we assume the docs are already available and ready).
- ❖ Architecture of solutions.
- ❖ Distribution of tasks to the team.

Where the solutions architecture are as below:

Default Odoo v17.0 Enterprise

Well, using default Odoo v17.0 Enterprise bank import statement modules and server actions and cron jobs, the company can receive the bank statements with the transactions via email, then set it up in Odoo where when it fetches from the configured incoming server it can:

- Agree with the banks on the structure of the email so that Odoo can scan new mails using domains to get the right email if the company is using one incoming server to handle multiple emails instead of a specific configured incoming server (i.e., specific is best and easy to handle).
- Set custom fields in account.payment / payment.transaction / ir.cron, e.g., last_fetch_date, stmt_has_been_processed on models that will be datetime and booleans just to make sure we don't process the same bank statement twice.
- Through the cron job code, get the attachment and check whether it's OFX or CSV based on the last check, making sure we don't re-process previous statements but only the latest statements fetched.
- Through the cron / server actions code, handle and process the OFX, CSV files from the newly fetched statement.
- Call the relevant processing code for OFX or CSV depending on which is the right one.
- Process the bank statement through the right module code, validating and posting the related invoice and its sale order.
- Use message posts (i.e., chatter) to notify the right parties at each stage.
- In peak season we can have a res.config.settings in Invoice, e.g., Auto process Sale Orders, where when the accountant is not available we set it on and adjust the cron job code to only work when this option is set. Alternatively, we can deactivate the cron and its server actions.

Custom modules v17.0

So here we basically need a module or changes which will allow automation of the sale order process by:

- First, getting to know whether the bank that's being used for payment has an API for handling payments. Most banks today do, so we need to understand the API and see whether there is already an existing module that does this and is either open source or cheap to purchase and use.
- If there isn't any cheap or open source module, then we create a custom payment provider module, which is

basically a module to handle payment processes done to that bank.

- Normally the new module(s) implementation will follow:
- Making use of existing code in payment, account_payment, sale, sale_management, account, etc., modules in Odoo v17.0.
- The normal naming will be payment_*bank*, where 'bank' is the bank provider API we are implementing, e.g., payment_abn_amro for ABN AMRO bank API.
- Add the bank as a payment provider and have configurations for its API, e.g., authentication used.
- When the customer has selected the items in the cart and is at the checkout page, they select the payment provider they use.
- When they do this, a payment transaction related to the order is created in draft state, then the customer is redirected to the bank's checkout page, but internally the below is happening in our code:
    - ❖ Configuration of order and payment provider information which mainly includes:
        - ➔ Return callback function to redirect customers to Odoo from the bank checkout page.
        - ➔ Notify URL to receive transaction info from the bank for post-processing.
        - ➔ API authentication, e.g., generated tokens, etc.
        - ➔ Company bank details.
        - ➔ Customer bank details.
        - ➔ Order details: reference, amount, etc.
    - ❖ This can be done internally in code as a POST sent, or a part of it is sent to trigger the bank's checkout page, which contains the order details, a specific auth pattern, reference, and amount. Then the customer is redirected to the bank's online checkout page where they enter either bank details or Visa/Mastercard details and submit for final order processing.
    - ❖ The URL callbacks:
        - ➔ **Notify URL**: This will handle the transaction information from the bank (whether it was successful, error occurred, pending, etc.), which will be updated on the payment transaction related to the order.
        - ➔ **Callback URL**: This is where the customer is redirected back to Odoo and can now see the status info from the notified URL if it was successful or not with the right information.
- The customer is redirected back to Odoo where they can view the right status information depending on the post information sent by the bank to Odoo.
- When it is successful, a confirmation page showing the order details and a PDF print of the Odoo order is shown to the customer where they can choose to continue shopping or end the shop and await the order to be processed and delivered.
- After that, in the backend the payment transaction is sent from draft to the right state (here let's use "successful") and then code for creating a payment is made, but normally you would use Odoo v17.0 code that already exists that will call the right functions for posting the payment, i.e., _setup_provider, _create_payment_transaction, _get_processing_values, _get_specific_rendering_values, _handle_notification_data, _process_notification_data, _set_done, _update_state, _execute_callback, _cron_finalize_post_processing, _log_sent_message, _create_payment, etc. There are many other functions I

haven't referenced but are essential, like functions to handle errors, status, batch payment processing, etc.

- So after the payment transaction is updated and payment is recorded, we can validate and post the associated invoice and its related sale order as we notify the relevant entities at each stage.
- In some parts we can set the functions to use the OCA queue_job module, e.g., invoice validation, payment processing, etc.
- We can go a step further into processing the outgoing stock pickings etc..

How much time for giving an answer/development?

The time taken to do this can be broken into phases and their approximate times: - Investigation and research of possible solutions: ~1hr 30mins - Architecture of selected solutions & distribution of tasks: ~1hr 30mins - Implementation of solutions: ~2hrs - Testing and Code Review: ~1hr 30mins - Deployment: ~40mins

Please note I gave approximations, where we can give an answer within ~1hr and 30mins of investigating and researching solutions, and complete architecture, implementation, testing, code review, and deploying the fix can take the other 5hrs 40mins, leading close to a day's work. With the aid of A.I. tools where applicable, we can narrow down that period by several hours.

Case 2 Answers

How are we going to handle this?

Handling this is not a big issue in Odoo v17.0 Enterprise because Odoo provides all the necessities to manage this without a problem. You can practically handle this using just default Odoo with its default features, server actions, and cron jobs. So to handle this you will first need to set up custom fields using Odoo Studio:

- Field **"Vendor Products Supplied"** Type: Many2Many - we set all the products a vendor supplies to the company.
- Field **"Does Vendor meet 25% Sold goods threshold"** Type: Boolean - we set it if the current vendor based on the sales done on their products meets the 25% threshold in that year.

Then you design the following server actions, automation rules, and cron job:

---

1. Update vendor products list

Model

```
res.partner
```

Name

```
Update vendor products list
```

Purpose

- Shows, on the vendor record, all products that vendor supplies.
- Makes the vendor/product relationship easier to explain and demo.
- Odoo stores this relationship normally on the product side through vendor pricelist lines, so this action creates a vendor-side view of that data.

Custom field used

```
x_studio_many2many_field_vendor_products
```

Field model

```
res.partner
```

Design features

- Can run on one vendor or multiple selected vendors.
- Searches products where the vendor appears in product.supplierinfo.
- Supports commercial partner logic, so child contacts and parent vendor companies are handled.
- Can optionally filter:
    - ❖ only purchasable products,
    - ❖ only storable products,

- archived or active products.

- Writes the result into the vendor many-to-many field.
- Posts a note on the vendor showing how many products were assigned.

---

2. Check vendor sold goods threshold

Model

```
res.partner
```

Name

```
Check Vendor Sold Goods Threshold
```

Purpose

- Checks whether a vendor qualifies for the focused pricelist/replenishment workflow.
- Helps identify vendors where enough supplied products are actually being sold.

Custom field used

```
x_studio_does_vendor_meet_25_sold_goods_threshold
```

Field model

```
res.partner
```

Design features

- Searches all products supplied by the vendor.
- Reads confirmed sales order lines for those products.
- Counts how many supplied products were sold.
- Calculates sold product percentage.
- Sets the threshold boolean to True or False.
- Posts a note on the vendor with:
  - sales year checked,
  - vendor product count,
  - sold product count,
  - sold product percentage,
  - whether the threshold passed.

---

3. Create reordering rules for sold vendor products

Model

```
res.partner
```

Name

```
Analyze Sold Vendor Products and Create Reordering Rules
```

Purpose

- Creates replenishment rules only for vendor products that are actually selling.
- Avoids creating reorder automation for the whole vendor catalog.

Design features

- Runs from the vendor.
- Searches all products supplied by the vendor.
- Checks confirmed sales for those products.
- Ranks products by:
    - ❖ quantity sold,
    - ❖ sales amount,
    - ❖ or number of sale lines.
- Selects the top sold products.
- Checks whether each selected product already has a reordering rule.
- Creates missing reordering rules.
- Updates existing reordering rules if enabled.
- Uses the warehouse stock location.
- Uses the Buy route where available.
- Uses the product's vendor pricelist/supplierinfo line.
- Can use manual or automatic reordering triggers.

Recommended demo setting

```
Trigger = Manual
```

Why manual first

- Easier to explain.
- Safer for demo.
- Lets purchase review the replenishment proposals before RFQs are created automatically.

Business message

- Vendor pricelist gives the purchase price.
- Sales history decides which products matter.
- Reordering rules decide which products Odoo should replenish.

4. Request vendor pricelist for sold products

Model

```
res.partner
```

Name

```
Request Vendor Pricelist for Sold Products
```

Purpose

- Generates an email to the vendor asking for updated purchase prices.
- Only requests prices for products from that vendor that were actually sold.

Guard condition

```
x_studio_does_vendor_meet_25_sold_goods_threshold = True
```

Design features

- Runs from one selected vendor.
- Checks that the vendor passed the sold goods threshold.
- Searches products supplied by the vendor.
- Checks confirmed sales history.
- Selects relevant sold products.
- Builds an email table with product details.
- Opens the Odoo email composer.
- Uses a subject starting with:

```
Pricelist:
```

Example subject

```
Pricelist: Request for 2027 purchase prices - Case2 Technical Vendor 01
```

Email asks vendor for

- updated purchase price,
- currency,
- minimum order quantity,
- packaging or box quantity,
- expected lead time,
- discontinued/replacement product references.

Design reason

- The subject token Pricelist: helps the incoming email automation identify valid replies.

---

5. Process vendor pricelist CSV reply

Model

```
mail.message
```

Name

```
Process Vendor Pricelist CSV Reply
```

Purpose

- Processes the vendor's CSV reply and updates Odoo vendor pricelist lines.

Expected incoming message

- The related document model is res.partner.
- Subject contains Pricelist:.
- Message has a .csv attachment.
- Vendor threshold field is True.

CSV format

```
internal_reference;vendor_product_code;vendor_product_name;price;currency;min_qty;lead_time_days
```

Design features

- Checks that the message is linked to a vendor/contact.
- Checks that the subject is a pricelist reply.
- Checks that the vendor meets the sold goods threshold.
- Finds CSV attachments.
- Reads the CSV attachment.
- Validates the full CSV before updating anything.
- Matches products by:
    - ❖ internal reference first,
    - ❖ vendor product code as fallback.
- Finds or creates the vendor pricelist/supplierinfo line.
- Updates:
    - ❖ vendor product code,
    - ❖ vendor product name,
    - ❖ purchase price,
    - ❖ Currency,
    - ❖ minimum quantity,

        - ❖ delivery lead time.
- Posts a success note on the vendor.
- Posts a note on each updated product.
- If the CSV is invalid:
    - ❖ no lines are updated,
    - ❖ a note is posted on the vendor,
    - ❖ an error email can be sent back to the vendor with the required CSV format.

Tested result

- 3 vendor pricelist lines updated.
- 0 vendor pricelist lines created.
- 3 product notes posted.
- 1 CSV attachment processed.
- Product Purchase tab showed updated vendor price and lead time.

That action is important because it is the bridge between:

```
Vendor passed threshold
Selected/sold products identified
Vendor receives a structured email table
Vendor replies with CSV pricelist/restocking information
```

6. Generate vendor restocking/pricelist request email

Model

```
res.partner
```

Name

```
Request Vendor Pricelist for Sold Products
```

Purpose

- Generates an email to the vendor.
- Includes a table of the vendor's products that are relevant for restocking.
- Only works for vendors that passed the sold-goods threshold.
- Asks the vendor to send back updated prices, minimum quantities, lead times, and availability in CSV format.

Custom field checked

```
x_studio_does_vendor_meet_25_sold_goods_threshold
```

Design features

- Runs from the vendor form.

- Checks whether the vendor passed the threshold.
- Searches all products supplied by that vendor.
- Checks which of those products have confirmed sales.
- Ranks or filters the sold products.
- Builds an email table of the relevant products.
- Opens the Odoo email composer.
- Uses a controlled subject starting with:

```
Pricelist:
```

Example subject

```
Pricelist: Request for 2027 purchase prices – Case2 Technical Vendor 01
```

The email table includes

- Internal Reference.
- Product name.
- Vendor product code.
- Vendor product name.
- Quantity sold in the selected year.
- Current vendor price.
- Current minimum quantity.
- Current delivery lead time.

The email asks the vendor to provide

- Updated purchase price.
- Currency.
- Minimum order quantity.
- Packaging or box quantity.
- Expected lead time.
- Replacement product references if a product is discontinued.
- Availability/restocking information if relevant.

Why this action matters

- It avoids asking the vendor for the full product catalog again.
- It focuses only on products that matter commercially.
- It gives the vendor a clear table to respond to.
- It prepares the vendor reply for the next automation step.
- It makes the CSV update process predictable.

Output

- Opens an email composer.
- Posts a note on the vendor saying the email was prepared.
- The vendor can then reply with a CSV attachment.

---

7. Update vendor unsold products list

Model

```
res.partner
```

Name

```
Update vendor unsold products list
```

Purpose

- Checks the products supplied by a vendor.
- Finds which supplied product variants have not been sold in the selected year.
- Marks those product variants as not meeting the vendor sold threshold.
- Links the unsold product variants back to the vendor in a many-to-many field.
- Helps users filter products that should not be included in automatic replenishment.

Custom fields used

```
x_studio_does_not_meet_vendor_sold_threshold_25_1
x_studio_vendor_products_25_unsold_ids
```

Field models

```
product.product
res.partner
```

Design features

- Runs from the vendor form.
- Can run on one vendor or multiple selected vendors.
- Searches all product variants supplied by the selected vendor.
- Uses product.supplierinfo to know which products belong to the vendor.
- Checks confirmed sales order lines for the selected year.
- Splits the vendor's product variants into:
    - ❖ sold product variants,
    - ❖ unsold product variants.
- Sets the product boolean to True for unsold variants:

```
x_studio_does_not_meet_vendor_sold_threshold_25_1 = True
```

- Sets the product boolean to False for sold variants:

```
x_studio_does_not_meet_vendor_sold_threshold_25_1 = False
```

- Writes the unsold product variants into the vendor many-to-many field:

```
x_studio_vendor_products_25_unsold_ids
```

- Supports filtering only:
  - ❖ purchasable products,
  - ❖ storable products,
  - ❖ active products.
- Posts a note on the vendor with:
  - ❖ sales year checked,
  - ❖ total supplied product variants,
  - ❖ sold product variants,
  - ❖ unsold product variants,
  - ❖ sold percentage,
  - ❖ unsold percentage,
  - ❖ field updated.

Why this action matters

- It gives the business a clear list of vendor products that are not selling.
- It helps avoid creating reordering rules for products that do not move.
- It supports filtering product variants where the unsold threshold boolean is set.
- It gives the vendor record a direct view of products that should not drive replenishment.
- It separates the full vendor catalog from the active replenishment assortment.

Output

- Updates the product variant boolean.
- Updates the vendor many-to-many field.
- Posts a note on the vendor with the result.
- Allows users to filter unsold products directly from product variants or vendors.

---

8. Cron: Update vendor unsold products 25 threshold

Model

```
res.partner
```

Name

```
Cron: Update vendor unsold products 25 threshold
```

Purpose

- Automatically refreshes the unsold-product analysis for all vendors.
- Keeps the product boolean and vendor many-to-many fields up to date.
- Avoids manually running the unsold-product server action vendor by vendor.
- Helps maintain clean replenishment decisions over time.

Custom fields used

```
x_studio_does_not_meet_vendor_sold_threshold_25_1
x_studio_vendor_products_25_unsold_ids
```

Suggested frequency

```
Monthly
```

Design features

- Runs as a scheduled action.
- Searches all vendors used in product.supplierinfo.
- Checks all product variants supplied by each vendor.
- Reads confirmed sales order lines for the selected year.
- Detects which vendor product variants were sold.
- Detects which vendor product variants were not sold.
- Sets the unsold product boolean on product variants:

```
x_studio_does_not_meet_vendor_sold_threshold_25_1
```

- Updates the vendor many-to-many field:

```
x_studio_vendor_products_25_unsold_ids
```

- Clears the boolean on product variants that have been sold.
- Can be limited to process a certain number of vendors per run.
- Can be configured to check only:
    - ❖ purchasable products,
    - ❖ storable products,
    - ❖ active products.
- Posts a note on each processed vendor with:
    - ❖ sales year checked,
    - ❖ total supplied product variants,
    - ❖ sold product variants,
    - ❖ unsold product variants,
    - ❖ sold percentage,
    - ❖ threshold reference.

- Logs a summary of the cron result.

Why this cron matters

- It keeps vendor product analysis current without manual work.
- It helps purchasing teams identify products that should not be replenished.
- It prevents unsold products from being treated the same as active selling products.
- It supports better filtering and reporting.
- It keeps the vendor/product data useful throughout the year.

Output

- Updates unsold product flags on product variants.
- Updates the vendor many-to-many field with unsold products.
- Posts a vendor note after each update.
- Logs the cron result for review.

Automation Rule

1. Auto handle vendor email with CSV

Model

```
mail.message
```

Name

```
Auto Handle Vendor Email With CSV
```

Trigger

```
After creation
```

Apply on

```
Related Document Model = res.partner
Attachments is set
Subject contains Pricelist:
```

Action

```
Execute Existing Actions
```

Child action

```
Process Vendor Pricelist CSV Reply
```

Purpose

- Automatically processes vendor pricelist replies when Odoo receives or creates the message.

- Avoids manually running the CSV processor.

Design features

- Runs only when a new message is created.
- Filters only vendor/contact messages.
- Requires an attachment.
- Requires subject to contain Pricelist:.
- Then delegates the actual CSV processing to the server action.

Why "After creation"

- The incoming email creates a new mail.message.
- We want to process the message once after it is created.
- We avoid "On save" because that may run again if the message is later updated.

Business Process Flow

With the server actions defined above we can say the process flow is as below:

Vendor product setup

- Run **Update vendor products list**.
- Odoo finds all products supplied by the vendor.
- Odoo writes those products to the vendor many-to-many field.
- Odoo posts a note on the vendor.

Vendor sales threshold check

- Run **Check Vendor Sold Goods Threshold**.
- Odoo checks which supplied products were sold.
- Odoo updates:

```
x_studio_does_vendor_meet_25_sold_goods_threshold
```

- Odoo posts a note on the vendor.

Reordering rule creation

- Run **Analyze Sold Vendor Products and Create Reordering Rules**.
- Odoo finds sold vendor products.
- Odoo creates or updates reordering rules.
- Only products with real sales history get replenishment rules.

Restocking/pricelist request email

- Run **Request Vendor Restocking Pricelist**.
- Odoo checks that the vendor passed the threshold.
- Odoo builds an email table of relevant sold/restocking products.
- Odoo asks the vendor for updated prices and restocking details.
- The email subject starts with Pricelist:.
- Odoo posts a note on the vendor.

Vendor reply

- Vendor replies with a CSV attachment.
- Subject keeps Pricelist:.

Incoming CSV automation

- Automation rule runs on mail.message.
- It checks:
  - ❖ related model is res.partner,
  - ❖ attachment exists,
  - ❖ subject contains Pricelist:.

CSV processing

- Odoo validates the CSV.
- Odoo matches rows to products and supplierinfo.
- Odoo updates vendor pricelist lines.
- Odoo posts a note on the vendor.
- Odoo posts a note on each updated product.
- If the CSV is wrong, Odoo emails the vendor back with the required format.

How is the process working?

The current process based on the description seems to be working manually, thus giving the impression that managing pricelist data from the vendors for the 45K products is a cumbersome issue, but it's quite easy once the right configurations and server actions are set up. First of all, Odoo v17.0 has reorder rules which help you manage automation of product reorder quantities by auto-generating purchase orders on them once certain minimum quantities are reached. From there then you can manually send the purchase orders to the vendor requesting more quantities of the products. The pricelists are manually handled where they are created detailing each product and their various prices, so you can import the pricelist or manually add them depending on which method is preferred.

Custom code or not?

As defined in the question **"How are we going to handle this?"** we can see that you can manage this automation process using just server actions, cron jobs, and maybe automation rules. If you have to make custom code, it would just be to create the fields, server actions, and cron jobs mentioned into a module so that they persist in case of upgrades or modifications. That's the only custom modification I see, to keep data persistent in a module upon upgrades and database modifications.

Things to keep away from or not

Things to keep away from would be:

- **Misconfigurations**: Avoid configuring Odoo settings and data the wrong way, which will lead to data integrity issues.
- **Product data duplications**: Out of the 45K products you need to avoid duplicates, which will lead to wrong data updates and integrity issues.
- **Complex customizations**: An example of complex customizations would be allowing the vendor to modify the pricelist through an external portal. If the vendor feeds wrong values, it affects data integrity.

Things to maintain and add

- **Access groups and access rights**: Manage actions through access groups where actions cannot be accessed by everybody but just a select few or the right group.
- **Vacuum/Analyze tables**: Not used to avoid a large database with unused data through PostgreSQL.
- **Use existing features and modules**: Before development and customization becomes an option, try using Odoo default features and also check well-maintained modules like OCA for any feature you might need, e.g., OCA product-attribute repo, purchase-workflow, etc.
- **Auditing & Logging**: You can use Odoo logging and profiler to monitor features and issues if any arise. Another good module would be OCA auditlog which allows the administrator to log user operations performed on data models such as create, read, write and delete.

Performance issues yes/no?

No, currently at 45K products and some are not being used and only 25% of a vendor's products incur sales, there aren't many performance issues. However, reordering rules are fine when created only for products that should actually be replenished. Updating vendor prices once per year is manageable if done in controlled batches, and if only the active/sold 25% is used for replenishment, the daily purchase flow stays reasonable.

Default Odoo, customization and/or Studio?

Both default Odoo and Studio as described in **"How are we going to handle this?"** question, where the flow will be as below:

Vendor product setup

- Run **Update vendor products list**.
- Odoo finds all products supplied by the vendor.
- Odoo writes those products to the vendor many-to-many field.
- Odoo posts a note on the vendor.

Vendor sales threshold check

- Run **Check Vendor Sold Goods Threshold**.
- Odoo checks which supplied products were sold.
- Odoo updates:

```
x_studio_does_vendor_meet_25_sold_goods_threshold
```

- Odoo posts a note on the vendor.

Reordering rule creation

- Run **Analyze Sold Vendor Products and Create Reordering Rules**.
- Odoo finds sold vendor products.
- Odoo creates or updates reordering rules.
- Only products with real sales history get replenishment rules.

Restocking/pricelist request email

- Run **Request Vendor Restocking Pricelist**.
- Odoo checks that the vendor passed the threshold.
- Odoo builds an email table of relevant sold/restocking products.
- Odoo asks the vendor for updated prices and restocking details.
- The email subject starts with Pricelist:.
- Odoo posts a note on the vendor.

Vendor reply

- Vendor replies with a CSV attachment.
- Subject keeps Pricelist:.

Incoming CSV automation

- Automation rule runs on mail.message.
- It checks:
  - related model is res.partner,
  - attachment exists,
  - subject contains Pricelist:.

CSV processing

- Odoo validates the CSV.
- Odoo matches rows to products and supplierinfo.
- Odoo updates vendor pricelist lines.
- Odoo posts a note on the vendor.
- Odoo posts a note on each updated product.
- If the CSV is wrong, Odoo emails the vendor back with the required format.

How much time for giving an answer/development?

The time taken to do this can be broken into phases and their approximate times:

- Investigation and research of possible solutions: ~1hr 30mins
- Architecture of selected solutions & distribution of tasks: ~1hr 30mins
- Implementation of solutions: ~2hrs
- Testing and Code Review: ~1hr 30mins
- Deployment: ~40mins

Please note I gave approximations, where we can give an answer within ~1hr and 30mins of investigating and researching solutions, and complete architecture, implementation, testing, code review, and deploying the fix can take the other 5hrs 40mins, leading close to a day's work. With the aid of A.I. tools where applicable, we can narrow down that period by several hours.

Other advice?

I would advise to first use and implement the solution using Odoo default, Studio, and its server actions and cron job automation. Then if need be, we can implement those solutions into a module so that at least we have a restore point in case data is changed in the database. For general healthy data on the system, it would be good to avoid data integrity issues like duplication of products, misconfigurations, unused server actions, cron jobs, etc. Another thing I would recommend is using OCA modules from their respective Odoo ERP module repos e.g purchase-workflow, purchase-reporting, product-attribute etc.